TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN
BASE DE DATOS II

Forma Normal de Boyce-Codd y desnormalización controlada en Food Store

Trabajo Práctico · Unidad 4
Normalización avanzada y redundancia

Materia	Base de Datos II
Unidad	Unidad 4 — Forma Normal de Boyce-Codd y Desnormalización Controlada
Trabajo	Parte 1: FNBC sobre ControlLoteAlmacen Parte 2: reporte top 5 de categorías por monto vendido
Integrantes	Fernando Torrez 53.988 Ignacio Sánchez 53.966 Agustín Contardi 53.863 Enzo Giaquinta 53.884 Ramiro Salcedo 53.992
Fecha de entrega	28 de octubre de 2026
Motor utilizado	PostgreSQL 17.11 (Windows x64)
Entregables	tp_fnbc_control_lote.sql · tp_desnormalizacion_top_categorias.sql · este informe

Alcance
Todas las afirmaciones técnicas de este informe están respaldadas por un artefacto: el script SQL que las implementa o la salida de **EXPLAIN (ANALYZE, BUFFERS)** que las mide. Las capturas completas de los planes de ejecución se incluyen en el Anexo B y, además, como archivos de texto en la carpeta *resultados/* de la entrega.

Resumen ejecutivo

El trabajo aborda dos decisiones de diseño sobre la base Food Store y, en ambos casos, la decisión se toma con evidencia y no por intuición: en la Parte 1 se analizan las dependencias funcionales de la relación *control_lote_almacen* y se determina, con la definición formal, que el esquema está en Tercera Forma Normal (3FN) pero **no** en Forma Normal de Boyce-Codd (FNBC). La dependencia violatoria es **ResponsableControlID → DepositoID**, cuyo determinante no es superclave. La relación se descompone sin pérdida en *responsable_deposito* y *control_lote_responsable*; una vista de compatibilidad reconstruye la relación original por reunión natural y la verificación de equivalencia devuelve **0 diferencias** sobre la instancia.

En la Parte 2 se mide la consulta del panel (top 5 de categorías por monto vendido en el día) sobre una réplica poblada con 250.000 pedidos y 1.250.000 líneas de detalle. La consulta original tarda una **mediana de 148,46 ms** y el nodo que domina el costo es un **recorrido secuencial paralelo sobre detalle_pedido** (1,25 millones de filas). Aun agregando el índice sobre la clave foránea que PostgreSQL no crea por defecto, el tiempo baja sólo a 4,54 ms y el costo sigue creciendo con el volumen del día. Con una tabla de agregación incremental por fecha y categoría, mantenida por disparadores, la misma consulta responde en **0,075 ms** leyendo 12 filas: **1.979 veces más rápido** que el esquema original y 60 veces más rápido que el esquema indexado.

El dato redundante no queda librado a la disciplina de la aplicación: cuatro disparadores y una función de apoyo lo actualizan dentro de la misma transacción, y una vista de auditoría recalcula el agregado desde la fuente de verdad y compara. La auditoría ejecutada sobre la base migrada devuelve **0 filas**; además, se comprueba que no es vacua provocando una desincronización artificial, que la vista detecta de inmediato.

Entregable	Contenido
tp_fnbc_control_lote.sql	Esquema original, instancia de ejemplo, demostración de anomalías, tablas descompuestas, vista de compatibilidad y migración verificada (Parte 1).
tp_desnormalizacion_ top_categorias.sql	Esquema y población de la réplica, medición inicial, estructura desnormalizada con sus disparadores, consulta del panel, auditoría y pruebas del mecanismo (Parte 2).
Informe_TP6_FNBC_ Desnormalizacion.pdf	Este documento: análisis, justificaciones, resultados y capturas.
resultados/	Capturas de EXPLAIN (ANALYZE, BUFFERS) antes y después, salidas de los scripts, auditoría y scripts de benchmark reproducibles.

Parte 1 · Análisis y aplicación de la FNBC: ControlLoteAlmacen

1.1 · Contexto de dominio y esquema inicial

Food Store extiende su operación hacia la distribución mayorista: incorpora depósitos propios y recibe de sus proveedores lotes de mercadería que deben pasar un control de calidad antes de ingresar al inventario. Cuando un lote es de gran volumen se fracciona y se controla simultáneamente desde más de un depósito. Por cada depósito que interviene en el control de un lote se designa un responsable de control a cargo de esa fracción. El relevamiento con el área de logística produjo dos reglas:

- **Regla 1.** Para un lote y un depósito interviniente dados, el responsable de control queda unívocamente determinado.
- **Regla 2.** Cada responsable de control pertenece, como dato maestro de la dotación de personal, a un único depósito: no controla lotes coordinados desde depósitos distintos.

El esquema inicial que traduce ese relevamiento, con la instancia de ejemplo del enunciado, es el siguiente:

```
CREATE TABLE control_lote_almacen (
    lote_id                BIGINT NOT NULL REFERENCES lote(id),
    deposito_id            BIGINT NOT NULL REFERENCES deposito(id),
    responsable_control_id  BIGINT NOT NULL REFERENCES usuario(id),
    PRIMARY KEY (lote_id, deposito_id)
);

INSERT INTO control_lote_almacen VALUES
    (501, 30, 801),
    (502, 30, 801),
    (503, 31, 802);
```

lote_id	deposito_id	responsable_control_id
501	30	801
502	30	801
503	31	802

1.2 · (a) Dependencias funcionales en notación formal

Con los nombres de atributo indicados —LoteID (L), DepositoID (D) y ResponsableControlID (R)—, las dos reglas de negocio se traducen en:

DF	Expresión formal	Origen en la regla de negocio
DF1	(LoteID, DepositoID) → ResponsableControlID	Regla 1: para un lote y un depósito interviniente, el responsable queda unívocamente determinado.
DF2	ResponsableControlID → DepositoID	Regla 2: cada responsable pertenece a un único depósito.

El conjunto de dependencias relevante es **F = { (LoteID, DepositoID) → ResponsableControlID ; ResponsableControlID → DepositoID }**. De DF2 se deduce, por aumento (augmentation), la dependencia (LoteID, ResponsableControlID) → DepositoID, que resultará clave para justificar la reunión sin pérdida. No existe ninguna dependencia que determine LoteID a partir de DepositoID ni de ResponsableControlID: un depósito interviene en muchos lotes y un responsable controla muchos lotes.

1.3 · (b) Clausuras y conjunto completo de claves candidatas

Para determinar las claves se calculan las clausuras de todos los subconjuntos que resultan candidatos razonables. Un subconjunto X es superclave si X+ contiene los tres atributos; es clave candidata si además es minimal (ningún subconjunto propio suyo es superclave).

Subconjunto X	Clausura X+	¿Superclave?	¿Minimal?
{L}	{L}	No	—

Subconjunto X	Clausura X+	¿Superclave?	¿Minimal?
{D}	{D}	No	—
{R}	{R, D}	No	—
{L, D}	{L, D, R}	**Sí**	**Sí → clave candidata**
{L, R}	{L, R, D}	**Sí**	**Sí → clave candidata**
{D, R}	{D, R}	No	—
{L, D, R}	{L, D, R}	Sí	No (contiene a {L, D})

El conjunto completo de claves candidatas es **{LoteID, DepositoID}** y **{LoteID, ResponsableControlID}**. La primera es la clave primaria declarada en el esquema; la segunda no está declarada y, sin embargo, también determina toda la tupla.

En consecuencia, **todos los atributos son primos**: LoteID, DepositoID y ResponsableControlID pertenecen al menos a una clave candidata. **No hay atributos no primos**. Este detalle es el que explica por qué la relación cumple 3FN y, aun así, no cumple FNBC.

1.4 · (c) Verificación formal de FNBC

La definición de FNBC es: una relación R está en FNBC si y sólo si, para toda dependencia funcional no trivial X → Y en F+, X es una superclave de R. Se analiza cada dependencia de F:

- **DF1: (LoteID, DepositoID) → ResponsableControlID.** El determinante es {L, D}, que es clave candidata y, por lo tanto, superclave. No viola FNBC.
- **DF2: ResponsableControlID → DepositoID.** Es no trivial (el lado derecho no está contenido en el izquierdo) y su determinante es {R}, cuya clausura es {R, D}: no contiene LoteID, de modo que **no es superclave**. Esta dependencia **viola FNBC**.

La dependencia violatoria es, entonces, **ResponsableControlID → DepositoID**. El determinante ResponsableControlID no alcanza para determinar toda la tupla: identifica unívocamente un depósito, pero no el conjunto de lotes que ese responsable controla.

> **Por qué el caso es interesante: 3FN no es FNBC**
> La relación **sí cumple 3FN**. En la dependencia ResponsableControlID → DepositoID el atributo determinado (DepositoID) es primo, y la definición de 3FN admite un determinante que no es superclave cuando el lado derecho es un atributo primo. La FNBC elimina esa excepción: exige que *todo* determinante sea superclave, sin importar si el atributo determinado es primo o no. Este esquema es, por eso, un contraejemplo mínimo de la diferencia entre ambas formas normales.

La violación no es sólo teórica: el esquema la acepta. La clave primaria (LoteID, DepositoID) impide repetir el par lote–depósito, pero no impide que un mismo responsable quede asociado a dos depósitos. El script de la Parte 1 lo demuestra insertando la fila (504, 32, 801) cuando 801 ya pertenecía al depósito 30: el motor acepta la fila y deja a 801 en dos depósitos, contradiciendo la Regla 2.

1.5 · (d) Anomalías que habilita la violación

Las tres anomalías clásicas se manifiestan sobre la propia instancia de ejemplo. Cada escenario se ejecuta en el script dentro de una transacción que se revierte, para no alterar los datos.

Anomalía de inserción. El esquema no tiene dónde guardar un dato maestro del responsable: sólo existe la relación de control. Para registrar que el responsable 803 pertenece al depósito 32 hay que inventar, además, un control sobre un lote que 803 todavía no controló. En la demostración se dio de alta el lote 504 y se insertó la fila (504, 32, 803): el dato maestro quedó “colgado” de un hecho transitorio que puede no haber ocurrido. Si el lote 504 no existiera, el alta del responsable sería imposible.

Anomalía de actualización. El responsable 801 aparece en dos filas —(501, 30, 801) y (502, 30, 801)— porque controla dos lotes del mismo depósito. Si 801 pasa a pertenecer al depósito 31, la actualización correcta exige modificar *todas* las filas donde aparece. El script actualiza una sola y el resultado es que 801 queda en

los depósitos 30 y 31 simultáneamente: la Regla 2 queda violada y el motor no lo impide, porque el esquema no tiene forma de expresarla.

Anomalía de borrado. La fila (503, 31, 802) es el único registro que afirma que el responsable 802 pertenece al depósito 31. Si se da de baja el control de ese lote —un hecho operativo que termina— se borra también el dato maestro de la dotación: después del DELETE no queda ninguna fila que permita saber a qué depósito pertenece 802. Se pierde información que no debía depender de ese control.

1.6 · (e) Descomposición sin pérdida y DDL resultante

Se aplica el algoritmo de descomposición visto en clase: ante una dependencia X → Y que viola FNBC, se calcula la clausura X+ y se reemplaza R por dos relaciones, R1 = X+ y R2 = U − (X+ − X). Con X = ResponsableControlID:

```
X  = ResponsableControlID             (determinante de la DF violatoria)
X+ = {ResponsableControlID, DepositoID}

R1 = X+                         = {ResponsableControlID, DepositoID}
R2 = U - (X+ - X)               = {LoteID, DepositoID, ResponsableControlID} - {DepositoID}
                                = {LoteID, ResponsableControlID}
```

El script completo de las tablas resultantes, con claves primarias y claves foráneas, es:

```
CREATE TABLE responsable_deposito (
    responsable_control_id BIGINT PRIMARY KEY
        REFERENCES usuario(id),
    deposito_id            BIGINT NOT NULL
        REFERENCES deposito(id)
);

CREATE TABLE control_lote_responsable (
    lote_id                BIGINT NOT NULL
        REFERENCES lote(id),
    responsable_control_id BIGINT NOT NULL
        REFERENCES responsable_deposito(responsable_control_id),
    PRIMARY KEY (lote_id, responsable_control_id)
);

-- Vista de compatibilidad: reconstruye la relación original por reunión natural
CREATE VIEW control_lote_almacen_v AS
SELECT clr.lote_id,
       rd.deposito_id,
       rd.responsable_control_id
FROM control_lote_responsable clr
NATURAL JOIN responsable_deposito rd;
```

Ambas relaciones resultantes están en FNBC. En *responsable_deposito* el único determinante no trivial es ResponsableControlID, que es su clave primaria. En *control_lote_responsable* no hay dependencias no triviales: un lote puede tener varios responsables y un responsable puede controlar varios lotes, de modo que ninguno de los dos atributos determina al otro; su única clave, (LoteID, ResponsableControlID), no compite con ningún determinante.

1.7 · (f) Por qué la reunión es sin pérdida

El teorema de descomposición sin pérdida establece que una descomposición de R en R1 y R2 es sin pérdida si el atributo común de ambas relaciones es superclave de al menos una de ellas. En este caso:

```
R1 = responsable_deposito       = {ResponsableControlID, DepositoID}
R2 = control_lote_responsable    = {LoteID, ResponsableControlID}

R1 ∩ R2 = {ResponsableControlID}

Como ResponsableControlID -> DepositoID pertenece a F+,
ResponsableControlID es superclave de R1 (de hecho, es su clave primaria).
=> La reunión natural de R1 y R2 es sin pérdida.
```

La verificación empírica confirma la demostración: la vista de compatibilidad reconstruye las tres filas de la instancia original y la diferencia simétrica entre ambas relaciones es vacía (0 filas).

Una observación de rigor: preservación de dependencias
Las proyecciones de F sobre las dos tablas son F1 = {ResponsableControlID → DepositoID} y F2 = {}. Su unión no implica (LoteID, DepositoID) → ResponsableControlID, es decir, la descomposición **no preserva** esa dependencia. Es un resultado conocido: no toda relación admite una descomposición en FNBC que sea a la vez sin pérdida y preservadora de dependencias. Como el objetivo de la Parte 1 es la FNBC, se acepta esa pérdida y se recupera la regla de negocio con un trigger de restricción diferido sobre *control_lote_responsable*, incluido en el script (sección 5.7): impide que un mismo lote tenga dos responsables en el mismo depósito.

1.8 · (f) Migración de datos y verificación de equivalencia

La migración proyecta la instancia original sobre cada tabla resultante. En R1 se usa DISTINCT porque la proyección elimina, precisamente, la redundancia que el esquema original almacenaba: 801 aparecía dos veces asociado al depósito 30.

```
INSERT INTO responsable_deposito (responsable_control_id, deposito_id)
SELECT DISTINCT responsable_control_id, deposito_id
FROM control_lote_almacen;

INSERT INTO control_lote_responsable (lote_id, responsable_control_id)
SELECT lote_id, responsable_control_id
FROM control_lote_almacen;
```

Resultado de la verificación ejecutada por el script:

```
          verificacion           |     resultado     | diferencias
---------------------------------+-------------------+-------------
 equivalencia vista vs. original | OK: 0 diferencias |           0

            objeto            | filas
------------------------------+-------
 control_lote_almacen_orig    |     3
 responsable_deposito         |     2
 control_lote_responsable     |     3
 vista control_lote_almacen_v |     3
```

La vista reconstruye exactamente el ejemplo del enunciado: (501, 30, 801), (502, 30, 801) y (503, 31, 802). La relación original se conserva como *control_lote_almacen_orig* sólo como evidencia de la verificación; el estado final del esquema son las dos tablas en FNBC más la vista de compatibilidad.

Parte 2 · Consulta con desnormalización controlada

2.1 · (a) Medición inicial con EXPLAIN ANALYZE

Sobre la instancia utilizada
La Parte 2 se resuelve sobre una réplica de Food Store construida para este trabajo, con las mismas cuatro tablas del proyecto (*categoria*, *producto*, *pedido* y *detalle_pedido*). El script incluye el DDL y la población para que la medición sea reproducible; la sección 1 puede omitirse si se ejecuta sobre una copia real. La población es determinista (semilla fija y hashes) y tiene 250.000 pedidos distribuidos en 730 días, 800 productos y 1.250.000 líneas de detalle. El día de la medición registró 323 pedidos y 1.615 líneas.

Magnitud	Valor
Pedidos	250.000 (730 días; ~342 por día)
Líneas de detalle	1.250.000 (5 por pedido)
Productos / categorías	800 / 12
Pedidos y líneas del día medido	323 / 1.615
Tamaño de detalle_pedido	132 MB
Tamaño del resumen desnormalizado	1.072 kB (8.760 filas = 730 días × 12 categorías)
Configuración	shared_buffers 128 MB · work_mem 4 MB · 2 workers paralelos

La consulta del panel, tal como figura en el enunciado, produjo el siguiente plan (extracto; el plan completo está en el Anexo B.1):

```
Limit  (cost=20155.98..20155.99 rows=5 width=41) (actual time=120.162..129.587 rows=5 loops=1)
  Buffers: shared hit=12641
  ->  Sort  (cost=20155.98..20156.01 rows=12 width=41) (actual time=120.159..129.583 rows=5 loops=1)
        Sort Key: (sum(dp.subtotal)) DESC
        ->  Finalize GroupAggregate  ...
              ->  Gather Merge  ...
                    Workers Launched: 2
                    ->  Partial GroupAggregate  ...
                          ->  Hash Join  (Hash Cond: (pr.categoria_id = c.id))
                                ->  Hash Join  (Hash Cond: (dp.producto_id = pr.id))
                                      ->  Hash Join  (Hash Cond: (dp.pedido_id = ped.id))
                                            ->  Parallel Seq Scan on detalle_pedido dp
                                                  (actual time=0.013..60.776 rows=412494 loops=3)
                                                  Filter: (NOT eliminado)
                                                  Buffers: shared hit=11678
                                            ->  Hash  (rows=316)
                                                  ->  Bitmap Heap Scan on pedido ped
                                                        ->  Bitmap Index Scan on ix_pedido_fecha
Planning Time: 1.936 ms
Execution Time: 129.817 ms
```

Tiempo de ejecución: 129,82 ms en la captura de referencia; **mediana de 148,46 ms** sobre cinco ejecuciones. **Nodo que domina el costo:** el *Parallel Seq Scan* sobre *detalle_pedido*, que recorre las 1.250.000 filas (≈412.494 por worker) y consume 11.678 de los 12.641 buffers del plan.

La causa es estructural: el filtro por fecha está en *pedido*, no en *detalle_pedido*. El índice sobre *pedido(fecha)* reduce el lado del pedido a 325 filas, pero el motor no tiene forma de saber qué líneas pertenecen al día sin recorrer todas las líneas y unirlas con *pedido*. El agregado en sí es barato (1.567 filas sobreviven a los filtros y se agrupan en 12); lo caro es *encontrar* esas 1.567 filas dentro de 1,25 millones.

Para no atribuir a la desnormalización una mejora que en realidad produce un índice, se midió también el esquema con el índice sobre la clave foránea *detalle_pedido(pedido_id)*, que PostgreSQL no crea automáticamente. El plan cambia a un *Nested Loop* con 316 recorridos de índice y el tiempo baja a **4,54 ms** (mediana). Es una mejora de 32,7 veces, pero el costo sigue siendo proporcional a las ventas del día —se leen 1.567 líneas y 2.858 buffers por refresco— y se paga completo en cada actualización del panel.

2.2 · (b) Patrón elegido y justificación

Se eligió una **tabla de agregación incremental por fecha y categoría, mantenida por disparadores** (el equivalente a una columna precalculada, pero al grano exacto del reporte) y no una vista materializada: la evidencia de 2.1 muestra que el costo no está en acceder a una fila sino en recorrer y agregar ~1.600 líneas por refresco, de modo que lo que hay que evitar es la reagregación, no el acceso; el panel exige el dato del día en tiempo real, y una vista materializada sólo se actualiza al refrescarse (o exige *REFRESH ... CONCURRENTLY*, que recalcula el histórico completo o requiere un planificador externo), mientras que los disparadores aplican un delta sobre una única fila por operación, dentro de la misma transacción, con lo cual el resumen nunca queda desincronizado; y el patrón es reversible sin pérdida porque la tabla es 100 % derivada: puede reconstruirse desde la fuente de verdad con *fs_resumen_reconstruir()* o descartarse por completo sin perder un solo dato original.

2.3 · (c) Implementación

La estructura desnormalizada y su clave son:

```
CREATE TABLE resumen_venta_categoria_dia (
    fecha          DATE   NOT NULL,
    categoria_id   BIGINT NOT NULL REFERENCES categoria(id),
    total_vendido  NUMERIC(14,2) NOT NULL DEFAULT 0,
    cantidad_items BIGINT NOT NULL DEFAULT 0,
    actualizado_en TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (fecha, categoria_id)
);
```

El grano de la tabla es exactamente el del reporte: una fila por día y categoría. Es deliberado que **no** se copie el nombre de la categoría: se guarda *categoria_id* y el reporte une con *categoria* (12 filas). Así, un cambio de nombre no exige sincronizar nada y no se introduce una segunda fuente de verdad para el nombre.

La sincronización se reparte en cinco objetos, todos dentro de la misma transacción que la operación que los dispara:

Objeto	Evento	Qué hace
fs_resumen_aplicar	—	Upsert atómico: inserta o suma el delta sobre (fecha, categoría) con *ON CONFLICT*.
trg_detalle_resumen	INSERT / UPDATE / DELETE en detalle_pedido	Resta el aporte del estado anterior y suma el del estado nuevo, sólo si ni la línea ni el pedido están eliminados.
trg_pedido_resumen	UPDATE de fecha o eliminado en pedido	Resta el aporte del pedido en su estado anterior y suma el del nuevo (cambio de día o baja lógica).
trg_pedido_borrado	BEFORE DELETE en pedido	Resta el aporte antes de la cascada. Es BEFORE porque, cuando corre el borrado en cascada de detalle_pedido, el pedido ya no existe y el disparador de detalle no podría resolver la fecha.
trg_producto_categoria	UPDATE de categoria_id en producto	Mueve el histórico del producto: resta de la categoría anterior y suma en la nueva, por fecha.

La carga inicial (y la reconciliación) se resuelve con *fs_resumen_reconstruir()*, que trunca la tabla y la recalcula desde *detalle_pedido*, *producto* y *pedido*. La función es también el camino previsto para cargas masivas: *TRUNCATE* no dispara triggers de fila y un *COPY* histórico pagaría el costo de sincronización fila por fila, por lo que conviene deshabilitar los disparadores, cargar y reconstruir.

Para el mantenimiento del histórico se define además el procedimiento *fs_reconstruir_resumen(p_desde, p_hasta, p_commit, p_lote_dias)*, invocado con **CALL**. La diferencia con la función no es cosmética: el procedimiento recorre el rango en lotes —borra y recalcula cada bloque de días— y, si se le pide, confirma al terminar cada uno, de modo que una interrupción no obliga a empezar de nuevo. Sobre la base poblada, el histórico completo (730 días) se reconstruye en nueve lotes de 90 días en **2,55 s** y la auditoría vuelve a quedar en 0 filas (*resultados/call_procedimiento.txt*). El parámetro *p_commit* es *false* por omisión por una razón concreta: PostgreSQL no admite un *COMMIT* dentro de la transacción de otro, y un *CALL* con *p_commit = true* ejecutado dentro de un bloque *BEGIN* falla con «invalid transaction termination». La prueba de ambos

comportamientos está en la misma captura. El procedimiento queda fuera del camino caliente: no interviene en las ventas, sólo en las tareas de reconciliación.

2.4 · (d) Consulta sobre la estructura desnormalizada y comparación

La consulta del panel se reescribe para leer el agregado del día y unir sólo con la tabla de categorías:

```
SELECT c.nombre AS categoria,
       r.total_vendido
FROM resumen_venta_categoria_dia r
JOIN categoria c ON c.id = r.categoria_id
WHERE r.fecha = CURRENT_DATE
ORDER BY r.total_vendido DESC
LIMIT 5;
```

El plan resultante es un *Bitmap Index Scan* sobre la clave primaria del resumen (12 filas), un *Hash Join* con las 12 categorías y un *top-N heapsort*. No se toca *detalle_pedido*.

Métrica	Antes (consulta normalizada)	Después (consulta desnormalizada)
Tiempo de ejecución (mediana de 5 ejecuciones)	**148,46 ms** con el esquema original **4,54 ms** agregando el índice sobre la FK	**0,075 ms**
Nodo dominante	*Parallel Seq Scan* sobre detalle_pedido (1,25 M de filas) *Nested Loop* + *Index Scan* sobre detalle_pedido (316 iteraciones) en la variante indexada	*Bitmap Index Scan* sobre la PK del resumen (12 filas) + *Hash Join* con categoria
Filas leídas del detalle	1.250.000 (original) · 1.567 (indexada)	0
Buffers del plan	12.641 · 2.858	**19**
Mejora vs. esquema original	1×	**1.979×**

La tabla reporta la mediana de cinco ejecuciones para amortiguar el ruido del sistema operativo; las capturas completas de una ejecución representativa (129,82 ms / 10,48 ms / 0,16 ms) están en el Anexo B. La comparación más exigente es contra el esquema indexado, porque aísla el efecto de la desnormalización del efecto de un índice: aun así, el resumen es **60 veces** más rápido y lee 150 veces menos buffers.

La consulta desnormalizada devuelve exactamente los mismos totales que la normalizada, para las 12 categorías del día:

Categoría	Total normalizado	Total del resumen	¿Coincide?
Panadería	225.409,65	225.409,65	Sí
Limpieza	205.812,50	205.812,50	Sí
Lácteos	197.032,69	197.032,69	Sí
Frutas y Verduras	181.256,67	181.256,67	Sí
Congelados	178.898,80	178.898,80	Sí
… (7 categorías restantes)	…	…	Sí (12/12)

Los tipos NUMERIC hacen que la suma sea exacta: no hay diferencias por redondeo de punto flotante.

2.5 · (e) Auditoría de sincronización

La auditoría recalcula el agregado desde la fuente de verdad y lo compara con el resumen mediante un *FULL OUTER JOIN*, de modo que detecta tanto diferencias de monto como filas faltantes de un lado o del otro. Se implementó como vista para poder ejecutarla cuando haga falta:

```
CREATE OR REPLACE VIEW v_auditoria_resumen AS
WITH esperado AS (
    SELECT p.fecha, pr.categoria_id,
           SUM(dp.subtotal) AS total_vendido, SUM(dp.cantidad) AS cantidad_items
      FROM detalle_pedido dp
      JOIN producto pr ON pr.id = dp.producto_id
      JOIN pedido p    ON p.id  = dp.pedido_id
     WHERE NOT dp.eliminado AND NOT p.eliminado
     GROUP BY p.fecha, pr.categoria_id
)
SELECT COALESCE(e.fecha, r.fecha)               AS fecha,
       COALESCE(e.categoria_id, r.categoria_id) AS categoria_id,
       COALESCE(e.total_vendido, 0)             AS total_esperado,
       COALESCE(r.total_vendido, 0)             AS total_resumen,
       COALESCE(e.total_vendido, 0) - COALESCE(r.total_vendido, 0) AS diferencia,
       COALESCE(e.cantidad_items, 0)            AS items_esperados,
       COALESCE(r.cantidad_items, 0)            AS items_resumen
  FROM esperado e
  FULL OUTER JOIN resumen_venta_categoria_dia r
    ON r.fecha = e.fecha AND r.categoria_id = e.categoria_id
 WHERE COALESCE(e.total_vendido, 0) <> COALESCE(r.total_vendido, 0)
    OR COALESCE(e.cantidad_items, 0) <> COALESCE(r.cantidad_items, 0);
```

Ejecutada sobre la base ya migrada, la auditoría devuelve el resultado esperado:

```
 fecha | categoria_id | total_esperado | total_resumen | diferencia | items_esperados | items_resumen
-------+--------------+----------------+---------------+------------+-----------------+---------------
(0 filas)
```

Una auditoría que siempre devuelve vacío no demuestra nada por sí sola; por eso el script incluye una prueba de que **no es vacua**: dentro de una transacción se suma 1 al total de una categoría del día y la vista detecta exactamente una diferencia, que se revierte al hacer ROLLBACK. También se probaron las operaciones que mantienen el resumen (alta de línea, baja lógica de un pedido y recategorización de un producto): después de cada una, la auditoría sigue devolviendo 0 filas.

2.6 · Costo de escritura y operación

La desnormalización no es gratuita: cada línea escrita paga un upsert al resumen. Se midió insertando las 1.615 líneas del día con y sin el disparador, alternando el orden y repitiendo tres veces:

Operación (1.615 líneas)	Mediana	Por línea
INSERT con sincronización	179,0 ms	≈ 111 µs
INSERT con el disparador deshabilitado	31,7 ms	≈ 20 µs
Sobrecosto	≈ 5,6×	≈ 91 µs

Ese sobrecosto es aceptable para la relación lectura/escritura del panel: el reporte se ejecuta muchas veces por minuto y cada venta se escribe una vez. Para una carga histórica masiva, en cambio, conviene deshabilitar los disparadores, cargar y reconstruir con *fs_resumen_reconstruir()*.

En concurrencia, el *ON CONFLICT* toma un lock de fila sobre (fecha, categoría): dos ventas simultáneas de la misma categoría se serializan sólo en esa fila, no en toda la tabla. El resumen es, además, completamente reversible: *DROP TABLE resumen_venta_categoria_dia CASCADE* elimina la tabla y sus disparadores sin afectar un solo dato de la fuente de verdad.

Conclusiones

El caso de *control_lote_almacen* muestra con precisión dónde está la frontera entre 3FN y FNBC: como el atributo determinado por la dependencia violatoria es primo, la relación cumple 3FN y no cumple FNBC. La descomposición en *responsable_deposito* y *control_lote_responsable* elimina la redundancia y es sin pérdida —el atributo común es superclave de R1—, y la verificación sobre la instancia reconstruye las tres filas originales sin diferencias. El precio es que la dependencia (LoteID, DepositoID) → ResponsableControlID no se preserva; el trigger de restricción diferido la recupera a nivel de motor, lo que deja explícito el trade-off en lugar de esconderlo.

En la Parte 2, la medición desarma la intuición: la consulta no era lenta por el *GROUP BY* —agrupa 12 filas— sino por el recorrido secuencial de 1,25 millones de líneas para encontrar 1.567. Un índice sobre la clave foránea mejora 32,7 veces, pero no cambia la naturaleza del problema: el costo sigue creciendo con el volumen del día y se paga en cada refresco. El resumen por fecha y categoría desacopla el costo del volumen histórico y del volumen diario: 12 filas, 19 buffers, 0,075 ms.

La desnormalización se eligió con tres condiciones que el trabajo deja explícitas: está motivada por una medición, tiene un mecanismo de sincronización transaccional (no una ventana de inconsistencia), y es auditable y reversible. La auditoría devuelve vacío sobre la base migrada y, cuando se provoca una desincronización artificial, la detecta: el dato redundante no es una copia ciega, es un dato derivado con fuente de verdad, mecanismo de actualización y control de integridad.

Anexo A · Reproducción

Los scripts son idempotentes. La Parte 2 recrea el esquema base, por lo que debe ejecutarse sobre una copia o sobre una base vacía:

```
# 1) Base de trabajo
createdb -h localhost food_store_tp6

# 2) Parte 1: FNBC
psql -h localhost -d food_store_tp6 -v ON_ERROR_STOP=1 -f tp_fnbc_control_lote.sql

# 3) Parte 2: desnormalización (crea y puebla la réplica; ~30 s)
psql -h localhost -d food_store_tp6 -v ON_ERROR_STOP=1 -f tp_desnormalizacion_top_categorias.sql

# 4) Capturas y benchmark (opcional)
psql -h localhost -d food_store_tp6 -f resultados/benchmark.sql
psql -h localhost -d food_store_tp6 -f resultados/benchmark_explain.sql
psql -h localhost -d food_store_tp6 -f resultados/call_procedimiento.sql
```

Elemento	Detalle
Motor	PostgreSQL 17.11, Windows x64
Instancia usada para las mediciones	Clúster local aislado, puerto 55432, autenticación local
shared_buffers / work_mem	128 MB / 4 MB
Paralelismo	máximo 2 workers por consulta
Población	semilla 0.42 y hashes deterministas (reproducible)
Archivos de resultados	resultados/explain_antes.txt · explain_antes_optimizado.txt · explain_despues.txt · auditoria_final.txt · call_procedimiento.txt · benchmark_*.txt

Anexo B · Capturas de EXPLAIN (ANALYZE, BUFFERS)

B.1 · Antes: consulta original sobre el esquema normalizado (sin índice sobre la FK)

```
Limit  (cost=20155.98..20155.99 rows=5 width=41) (actual time=120.162..129.587 rows=5 loops=1)
  Buffers: shared hit=12641
  ->  Sort  (cost=20155.98..20156.01 rows=12 width=41) (actual time=120.159..129.583 rows=5 loops=1)
        Sort Key: (sum(dp.subtotal)) DESC
        Sort Method: top-N heapsort  Memory: 25kB
        Buffers: shared hit=12641
        ->  Finalize GroupAggregate  (cost=20147.36..20155.78 rows=12 width=41) (actual time=119.768..129.545 rows=12 loops=1)
              Group Key: c.nombre
              Buffers: shared hit=12638
              ->  Gather Merge  (cost=20147.36..20155.45 rows=24 width=41) (actual time=119.731..129.500 rows=36 loops=1)
                    Workers Planned: 2
                    Workers Launched: 2
                    Buffers: shared hit=12638
                    ->  Partial GroupAggregate  (cost=19147.34..19152.66 rows=12 width=41) (actual time=90.666..90.840 rows=12 loops=3)
                          Group Key: c.nombre
                          Buffers: shared hit=12638
                          ->  Sort  (cost=19147.34..19149.06 rows=689 width=15) (actual time=90.632..90.679 rows=522 loops=3)
                                Sort Key: c.nombre
                                Sort Method: quicksort  Memory: 47kB
                                Buffers: shared hit=12638
                                Worker 0:  Sort Method: quicksort  Memory: 40kB
                                Worker 1:  Sort Method: quicksort  Memory: 40kB
                                ->  Hash Join  (cost=870.23..19114.86 rows=689 width=15) (actual time=2.120..90.117 rows=522 loops=3)
                                      Hash Cond: (pr.categoria_id = c.id)
                                      Buffers: shared hit=12622
                                      ->  Hash Join  (cost=868.96..19111.15 rows=689 width=14) (actual time=1.858..89.654 rows=522 loops=3)
                                            Hash Cond: (dp.producto_id = pr.id)
                                            Buffers: shared hit=12589
                                            ->  Hash Join  (cost=843.96..19084.33 rows=689 width=14) (actual time=1.548..89.009 rows=522 loops=3)
                                                  Hash Cond: (dp.pedido_id = ped.id)
                                                  Buffers: shared hit=12568
                                                  ->  Parallel Seq Scan on detalle_pedido dp  (cost=0.00..16886.33 rows=515816 width=22) (actual time=0.013..60.776 rows=412494 loops=3)
                                                        Filter: (NOT eliminado)
                                                        Rows Removed by Filter: 4173
                                                        Buffers: shared hit=11678
                                                  ->  Hash  (cost=839.78..839.78 rows=334 width=8) (actual time=1.320..1.321 rows=316 loops=3)
                                                        Buckets: 1024  Batches: 1  Memory Usage: 21kB
                                                        Buffers: shared hit=890
                                                        ->  Bitmap Heap Scan on pedido ped  (cost=6.94..839.78 rows=334 width=8) (actual time=0.185..1.261 rows=316 loops=3)
                                                              Recheck Cond: (fecha = CURRENT_DATE)
                                                              Filter: (NOT eliminado)
                                                              Rows Removed by Filter: 7
                                                              Heap Blocks: exact=294
                                                              Buffers: shared hit=890
                                                              ->  Bitmap Index Scan on ix_pedido_fecha  (cost=0.00..6.86 rows=341 width=0) (actual time=0.068..0.068 rows=325 loops=3)
                                                                    Index Cond: (fecha = CURRENT_DATE)
                                                                    Buffers: shared hit=8
                                            ->  Hash  (cost=15.00..15.00 rows=800 width=16) (actual time=0.292..0.293 rows=800 loops=3)
                                                  Buckets: 1024  Batches: 1  Memory Usage: 46kB
                                                  Buffers: shared hit=21
                                                  ->  Seq Scan on producto pr  (cost=0.00..15.00 rows=800 width=16) (actual time=0.081..0.181 rows=800 loops=3)
                                                        Buffers: shared hit=21
                                      ->  Hash  (cost=1.12..1.12 rows=12 width=17) (actual time=0.220..0.221 rows=12 loops=3)
                                            Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                            Buffers: shared hit=3
                                            ->  Seq Scan on categoria c  (cost=0.00..1.12 rows=12 width=17) (actual time=0.205..0.207 rows=12 loops=3)
                                                  Buffers: shared hit=3
Planning:
  Buffers: shared hit=342
Planning Time: 1.936 ms
Execution Time: 129.817 ms
```

B.2 · Antes optimizado: el mismo esquema con índice sobre detalle_pedido(pedido_id)

```
Limit  (cost=8375.34..8375.36 rows=5 width=41) (actual time=10.478..10.483 rows=5 loops=1)
  Buffers: shared hit=2563 read=295
  ->  Sort  (cost=8375.34..8375.37 rows=12 width=41) (actual time=10.476..10.480 rows=5 loops=1)
        Sort Key: (sum(dp.subtotal)) DESC
        Sort Method: top-N heapsort  Memory: 25kB
        Buffers: shared hit=2563 read=295
        ->  HashAggregate  (cost=8375.00..8375.15 rows=12 width=41) (actual time=10.440..10.446 rows=12 loops=1)
              Group Key: c.nombre
              Batches: 1  Memory Usage: 24kB
              Buffers: shared hit=2560 read=295
              ->  Nested Loop  (cost=32.51..8366.74 rows=1652 width=15) (actual time=0.316..9.956 rows=1567 loops=1)
                    Buffers: shared hit=2560 read=295
                    ->  Hash Join  (cost=32.37..8325.04 rows=1652 width=14) (actual time=0.303..9.377 rows=1567 loops=1)
                          Hash Cond: (dp.producto_id = pr.id)
                          Buffers: shared hit=2536 read=295
                          ->  Nested Loop  (cost=7.37..8295.68 rows=1652 width=14) (actual time=0.119..8.840 rows=1567 loops=1)
                                Buffers: shared hit=2529 read=295
                                ->  Bitmap Heap Scan on pedido ped  (cost=6.94..839.78 rows=334 width=8) (actual time=0.090..1.015 rows=316 loops=1)
                                      Recheck Cond: (fecha = CURRENT_DATE)
                                      Filter: (NOT eliminado)
                                      Rows Removed by Filter: 7
                                      Heap Blocks: exact=294
                                      Buffers: shared hit=296
                                      ->  Bitmap Index Scan on ix_pedido_fecha  (cost=0.00..6.86 rows=341 width=0) (actual time=0.054..0.055 rows=325 loops=1)
                                            Index Cond: (fecha = CURRENT_DATE)
                                            Buffers: shared hit=2
                                ->  Index Scan using ix_detalle_pedido_pedido on detalle_pedido dp  (cost=0.43..22.27 rows=5 width=22) (actual time=0.013..0.024 rows=5 loops=316)
                                      Index Cond: (pedido_id = ped.id)
                                      Filter: (NOT eliminado)
                                      Rows Removed by Filter: 0
                                      Buffers: shared hit=2233 read=295
                          ->  Hash  (cost=15.00..15.00 rows=800 width=16) (actual time=0.171..0.171 rows=800 loops=1)
                                Buckets: 1024  Batches: 1  Memory Usage: 46kB
                                Buffers: shared hit=7
                                ->  Seq Scan on producto pr  (cost=0.00..15.00 rows=800 width=16) (actual time=0.011..0.086 rows=800 loops=1)
                                      Buffers: shared hit=7
                    ->  Memoize  (cost=0.15..0.18 rows=1 width=17) (actual time=0.000..0.000 rows=1 loops=1567)
                          Cache Key: pr.categoria_id
                          Cache Mode: logical
                          Hits: 1555  Misses: 12  Evictions: 0  Overflows: 0  Memory Usage: 2kB
                          Buffers: shared hit=24
                          ->  Index Scan using categoria_pkey on categoria c  (cost=0.14..0.17 rows=1 width=17) (actual time=0.002..0.002 rows=1 loops=12)
                                Index Cond: (id = pr.categoria_id)
                                Buffers: shared hit=24
Planning:
  Buffers: shared hit=354 read=6
Planning Time: 2.253 ms
Execution Time: 10.663 ms
```

B.3 · Después: consulta sobre la estructura desnormalizada

```
Limit  (cost=45.95..45.96 rows=5 width=17) (actual time=0.156..0.157 rows=5 loops=1)
  Buffers: shared hit=19
  ->  Sort  (cost=45.95..45.98 rows=14 width=17) (actual time=0.155..0.155 rows=5 loops=1)
        Sort Key: r.total_vendido DESC
        Sort Method: top-N heapsort  Memory: 25kB
        Buffers: shared hit=19
        ->  Hash Join  (cost=5.67..45.72 rows=14 width=17) (actual time=0.059..0.119 rows=12 loops=1)
              Hash Cond: (r.categoria_id = c.id)
              Buffers: shared hit=16
              ->  Bitmap Heap Scan on resumen_venta_categoria_dia r  (cost=4.40..44.40 rows=14 width=16) (actual time=0.030..0.086 rows=12 loops=1)
                    Recheck Cond: (fecha = CURRENT_DATE)
                    Heap Blocks: exact=13
                    Buffers: shared hit=15
                    ->  Bitmap Index Scan on resumen_venta_categoria_dia_pkey  (cost=0.00..4.39 rows=14 width=0) (actual time=0.020..0.020 rows=17 loops=1)
                          Index Cond: (fecha = CURRENT_DATE)
                          Buffers: shared hit=2
              ->  Hash  (cost=1.12..1.12 rows=12 width=17) (actual time=0.019..0.019 rows=12 loops=1)
                    Buckets: 1024  Batches: 1  Memory Usage: 9kB
                    Buffers: shared hit=1
                    ->  Seq Scan on categoria c  (cost=0.00..1.12 rows=12 width=17) (actual time=0.010..0.011 rows=12 loops=1)
                          Buffers: shared hit=1
Planning:
  Buffers: shared hit=262
Planning Time: 1.272 ms
Execution Time: 0.205 ms
```

B.4 · Auditoría de sincronización sobre la base migrada

```
 fecha | categoria_id | total_esperado | total_resumen | diferencia | items_esperados | items_resumen
-------+--------------+----------------+---------------+------------+-----------------+---------------
(0 filas)
```

Anexo C · Trazabilidad de la consigna

Parte 1 — 4.2

Punto	Dónde se responde	Evidencia en el script
a) Dependencias funcionales	§1.2	Comentario y sección 2
b) Clausuras y claves candidatas	§1.3	Consultas 2.1 a 2.4
c) Definición formal de FNBC	§1.4	Sección 3.4 (el motor acepta la violación)
d) Anomalías de inserción, borrado y actualización	§1.5	Sección 3.1 a 3.3
e) Descomposición, DDL y vista	§1.6	Secciones 4 y 5.3
f) Reunión sin pérdida y migración	§1.7 y §1.8	Secciones 5.1, 5.2 y 5.4

Parte 2 — 5.2

Punto	Dónde se responde	Evidencia en el script
a) EXPLAIN ANALYZE inicial	§2.1	Sección 3; resultados/explain_antes.txt
b) Patrón y justificación	§2.2	Sección 5
c) Implementación del patrón	§2.3	Sección 5 (tabla, disparadores y procedimiento de mantenimiento)
d) Consulta desnormalizada y comparación	§2.4	Sección 7; resultados/explain_despues.txt
e) Script de auditoría	§2.5	Sección 8 (v_auditoria_resumen)

Nota sobre el formato de entrega
El repositorio incluye los dos scripts SQL, este informe y la carpeta *resultados/* con las capturas completas y los scripts de benchmark. Las cifras del informe se obtuvieron de esas capturas; las medianas se calcularon sobre cinco ejecuciones para reducir el efecto del ruido del sistema operativo.